2025 UDPC

Official Solutions

UDPC 2025 - Writer / Tester

- ✓ Writer
  - minsung05
  - kwoncycle
  - 99asdfg
  - nukec
  - bnb2011
  - ekwoo
  - leo020630
  - slah007
  - andrew1010
  - queued_q
- ✓ Tester
  - jinhan814
  - jthis
  - playsworld16
  - tony9402
  - utilforever

Junior Division	Problem	Difficulty Intention	Writer
A	CPDU	**Easy**	leo020630
B	Porindrome	**Easy**	kwoncycle
C	Arkain Dashboard	**Easy**	queued_q
D	C)	**Medium**	leo020630
E	Guardian Angel Slime	**Medium**	leo020630
F	Gift Distribution	**Medium**	bnb2011
G	UDP String	**Hard**	99asdfg
H	Drone Light Show	**Hard**	queued_q
I	Treeo	**Hard**	leo020630

Senior Division	Problem	Difficulty Intention	Writer
A	CPDU	**Easy**	leo020630
B	Porindrome	**Easy**	kwoncycle
C	Arkain Dashboard	**Easy**	queued_q
D	Hamming Distance	**Medium**	queued_q
E	Guardian Angel Slime	**Medium**	leo020630
F	Gift Distribution	**Medium**	bnb2011
G	Drone Light Show	**Hard**	queued_q
H	Region Code	**Hard**	leo020630
I	C) and Query	**Hard**	leo020630

1A/2A. CPDU

`implementation, string`

Difficulty Intention – **Easy**

- ✓ Statistics:
  - Junior: 36 submissions, 22 correct, 61.111% AC rates
  - Senior: 22 submissions, 22 correct, 100.000% AC rates
- ✓ First Solve:
  - Junior: **yjun**[POSTECH], 0 min
  - Senior: **벌레캠프호날두**[POSTECH], 0 min
- ✓ Writer: Leo Jang[`leo020630`]

1A/2A. CPDU

- ✓ After receiving N strings as input, you simply need to check whether each string starts with the letter 'C'.

1B/2B. Porindrome

`bruteforcing, math`

Difficulty Intention – **Easy**

- ✓ Statistics:
  - Junior: 48 Submissions, 17 Accepted, AC rates - 35.417%
  - Senior: 62 Submissions, 22 Accepted, AC rates - 33.871%
- ✓ First Solve:
  - Junior: **outfinity**[UNIST], 5 min.
  - Senior: **과메기단**[UNIST], 4 min.
- ✓ Writer: Kwon Chan Woo[`kwoncycle`]

1B/2B. Porindrome

- First of all, numbers from 0 to 9 are all porindrome number.
- If the number with more than one digits is porindrome number, every digit of number should be same.
- Let N be a porindrome number with t digits, and define a_i be an ith number of N.
- By definition of porindrome number, $a_1a_2...a_t$ should be pelindrome, so $a_i = a_{t+1-i}$. Also, $a_1a_2...a_{t-1}$ should also be pelindrome, so $a_{i-1} = a_{t+1-i}$.
- This implies that for each i, $a_{i-1} = a_i$, which means that $a_1 = a_2 = ... = a_t$.
- Therefore, there are at most 10 kinds of porindrome number with same digit number, so the total number of porindrome number less than 10^9 is 82. We can simply count how many of them are less or equal than input N to solve problem.

1C/2C. Arkain Dashboard

`implementation, hash_set, sorting`

Difficulty Intention – **Easy**

- ✓ Statistics:
  - Junior: 37 Submissions, 14 Accepted, AC rates - 37.838%
  - Junior: 45 Submissions, 19 Accepted, AC rates - 42.222%
- ✓ First Solve:
  - Junior: **outfinity**[UNIST], 9 min.
  - Senior: **과메기단**[UNIST], 11 min.
- ✓ Writer: Han Dongkyu[queued_q]

- ✓ First, you need to know the last usage time for each container.
- ✓ Each time you get the i-th usage log from input, update the last usage time of that container to i.
  - `last_use[container] = i`
- ✓ Note that `container` is a string, not an integer index, so you need to use a hash table.
  - In C++, you can use `map` or `unordered_map`.
  - In Python, you can use `dict` (dictionary).

1C/2C. Arkain Dashboard

- ✓ Sort all containers in descending order of `last_use`.
- ✓ You can do this by making all `(last_use[container], container)` into pairs (C++) or tuples (Python), sorting them, then reversing the order.
- ✓ Alternatively, you can determine the order by checking the logs in reverse and skipping containers already checked.

- ✓ Now, let's separate the pinned containers from the unpinned containers.
- ✓ Check the sorted list in order, adding pinned containers to the `pinned` array and others to the `unpinned` array.
- ✓ You need to quickly determine whether each container is pinned or not.
  - Simply searching for the container name in the pinned containers list is slow, with a time complexity of $\mathcal{O}(K)$.
  - Note: The same applies when using the C++'s `find` function or Python's `in` operator.

- ✓ This can be resolved by storing the pinned container list in a hash table as well.
  - `is_pinned[container] = true`
- ✓ Storing and accessing values in a hash table is fast. (Time complexity = $\mathcal{O}(1)$)
- ✓ Check if a container is pinned as follows:
  - C++: `is_pinned[container]`
  - Python: `container in is_pinned`
- ✓ You can also use hash sets like `set` or `unordered_set`.
- ✓ Finally, combine the `pinned` array and `unpinned` array.
- ✓ The overall time complexity is $O(N \log N)$ or $O(N)$, depending on the implementation.

2D. C)

`greedy`

Difficulty Intention – **Medium**

- ✓ Statistics:
  - Junior: 39 submissions, 13 correct, 33.333% AC rates
- ✓ First Solve:
  - Junior: **outfinity**[UNIST], 17 min
- ✓ Writer: Leo Jang[`leo020630`]

2D. C)

- Using ’C’ as a closing parenthesis costs 2, while using ’U’ in either direction (opening or closing) costs 1.
- Therefore, we must minimize the number of ’C’ characters used as closing parentheses.
- If we determine the number of ’C’s to be used as closing parentheses, say k, it is optimal to use the k rightmost ’C’s as closing parentheses.
- Consequently, the number of ’U’s used as closing parentheses is determined as $\frac{|S|}{2} - k$, and it is also optimal to use the rightmost ’U’s as closing parentheses.

2D. C)

- Let’s repeat the following process:
- Initially, assume all ’C’s are opening parentheses and all ’U’s are closing parentheses.
- Define an opening parenthesis as -1 and a closing parenthesis as $+1$.
- Traverse the string from the end (backwards); at any moment the suffix sum becomes less than 0, that ’C’ must be used as a closing parenthesis.
- Through this method, we can determine the number of ’C’s to be used as closing parentheses, and the actual string can be constructed as described in the previous slide.

1D. Hamming Distance

`constructive`, `math`

Difficulty Intention – **Medium**

- ✓ Statistics:
  - Senior: 37 Submissions, 9 Accepted, AC rates - 24.324%
- ✓ First Solve:
  - Senior: **fm9779**[UNIST], 35 min.
- ✓ Writer: Han Dongkyu[queued_q]

1D. Hamming Distance

- ✓ Let's denote the two numbers we want to find as X and Y. $(A \leq X \leq Y \leq B)$
- ✓ Looking from the most significant bit of A and B, find the first position where they differ. Let's call the common bit sequence up to that position S.
- ✓ The most significant bit sequence of X and Y up to that position must be the same as S.
- ✓ Below that position, it is possible to make all the bits of X and Y different!

1D. Hamming Distance

- It can be constructed as follows.
  - $X = S0111\ldots1$
  - $Y = S1000\ldots0$
- Since the bit after S in A and B are different, they will be 0 and 1, respectively.
- X is the largest number starting with $S0$, and Y is the smallest number starting with $S1$, so they always satisfy $A \leq X \leq Y \leq B$.
- This solves the problem in $O(\log B)$ time if implemented properly.

1E/2E. Guardian Angel Slime

`sweeping`, `math`

Difficulty Intention – **Medium**

- Statistics:
  - Junior: 43 submissions, 10 correct, 23.256% AC rates
  - Senior: 36 submissions, 8 correct, 22.222% AC rates
- First Solve:
  - Junior: **outfinity**[UNIST], 37 min
  - Senior: **sg549**[DGIST], 48 min
- Writer: Leo Jang[`leo020630`]

1E/2E. Guardian Angel Slime

- Each slime will fall into one of two cases:
  - 1. It never reaches a size of X or greater.
  - 2. There exist two integers S and E ($S < E$) such that its size is X or greater during the interval $[S, E]$.
- Both determining the case and calculating S and E for the latter case can be done with simple formula calculations.

1E/2E. Guardian Angel Slime

- Once S and E are obtained for all applicable slimes, you can find the number of slimes with size $\geq X$ during specific date intervals by sorting these values and using prefix sums (or a sweeping technique).
- For each date interval, if the number of slimes with size $\geq X$ is 3 or more, add the interval length to the answer.
- The time complexity depends on sorting, which is $O(N \log N)$.
- Note that the number of possible days may exceed the range of a 32-bit integer.

1F/2F. Gift Distribution

`graph_traversal`

Difficulty Intention – **Medium**

- ✓ Statistics:
  - Junior: 22 Submissions, 6 Accepted, AC rates - 27.273%
  - Senior: 23 Submissions, 5 Accepted, AC rates - 21.739%
- ✓ First Solve:
  - Junior: **outfinity**[UNIST], 49 min.
  - Senior: **과메기단**[UNIST], 67 min.
- ✓ Writer: Shinwook Park[bnb2011]

1F/2F. Gift Distribution

- ✓ Interpret the conditions of the problem as follows:
  - Let's assume that assigning gift i to Yunee corresponds to value 0, and assigning it to Ponix corresponds to value 1 for vertex i.
  - Then, each condition U, D, and P corresponds to requiring the sum of the values assigned to vertices a_i and b_i to be 0, 1, and 2 respectively.
- ✓ In each connected component of the graph, once the value of one vertex is fixed, the values of all other connected vertices are automatically determined.
- ✓ Since the possible values are either 0 or 1, we try both cases and multiply the number of satisfying configurations to the total answer.
- ✓ The time complexity is $\mathcal{O}(N + M)$.

2G. UDP String

dp, combinatorics
Difficulty Intention – **Hard**

- ✓ Statistics:
  - Junior: 28 Submission, 8 Accepted, AC rates - 28.571%
- ✓ First Solve:
  - Junior: **equinox**[POSTECH], 72 min.
- ✓ Writer: Dohoon Kim[99asdfg]

2G. UDP String

- A complete UDP string cannot be created by concatenating any two UDP strings.
- By the definition of a UDP string, if a consecutive substring starting from the first index is a UDP string, then the other substring must also be a UDP string.
- Thus, the above condition is equivalent to the condition that every consecutive substring in the interval $[1, i]$ is not a UDP string for all $1 \leq i \leq 3N - 1$.
- Furthermore, by definition, the length of every UDP string is multiples of 3.
- We can make DP table using the conditions listed above.

2G. UDP String

- $DP[3i]$: the number of UDP strings such that the first occurrence of a UDP string in a consecutive substring appears in the interval $[1, 3i]$.
- $f(x)$: number of UDP strings of length $3x$
- By defining $DP[0] = 1$, we can derive the following formula.
- $f(x) = \frac{3x!}{(x!)^3}; dp[x] = f(x) - dp[1]f(x-1) - \cdots - dp[x-1]f(1)$
- Since $f(x)$ can be easily precomputed with a complexity of $\mathcal{O}\left(N^2\right)$, we can solve the entire problem by computing the DP values from 1 with a time complexity of $\mathcal{O}\left(N^2\right)$.
- You might use the modular inverse to quickly calculate $f(x)$ since its value could be extremely large.

1G/2H. Drone Light Show

`offline_queries, dp, graph_traversal`

Difficulty Intention – **Hard**

- ✓ Statistics:
  - Junior: 8 submissions, 6 correct, 75.000% AC rates
  - Senior: 8 submissions, 4 correct, 50.000% AC rates
- ✓ First Solve:
  - Junior: **van.li**[UNIST], 80 min
  - Senior: **벌레캠프호날두**[POSTECH], 102 min
- ✓ Writer: Dongkyu Han[queued_q]

1G/2H. Drone Light Show

- ✓ Simulating all server commands in order results in a time complexity of $O(MQ)$, which is too slow.
- ✓ Since all command information is available beforehand, let's utilize the idea of offline queries instead of processing them one by one.
- ✓ We will assign sequence numbers $1, 2, \ldots, Q$ to the commands.

1G/2H. Drone Light Show

- ✓ First, let's simplify the problem by assuming only downward-propagating commands are given.
- ✓ Let the drones directly connected to drone x from the upward direction be $y_1, \ldots, y_k$.
- ✓ After drone y_i executes a command, it will propagate that command to drone x.
- ✓ Therefore, the last command **executed** by drone x is the maximum index among:
  - The last command drone x **received directly from the server**, and
  - The last commands **executed** by drones y_i.
- ✓ This can be calculated using DP by traversing the drones from index N down to 1.

1G/2H. Drone Light Show

- ✓ Now, let's solve the problem considering that there are also upward-propagating commands.
- ✓ By performing DP separately for both directions, we can find the index of the very last downward command and the very last upward command executed by drone x.
- ✓ The color of the command with the larger index between the two will be the final color of drone x.
- ✓ The overall time complexity is $O(M + Q)$.
- ✓ Alternatively, the problem can be solved by performing a graph traversal for each direction in the reverse order of the queries.

1H. Region Code

`dp_bitfield`, `pigeonhole_principle`
Difficulty Intention – **Hard**

- Statistics:
  - Senior: 2 submissions, 0 correct, 0.000% AC rates
- First Solve:
  - Senior: -
- Writer: Leo Jang[leo020630]

1H. Region Code

- Let's consider the conditions under which a set of strings can be selected and rearranged character by character to form a palindrome.
- A palindrome can be formed if all characters appear an even number of times, or if exactly one character appears an odd number of times.
- Since the critical information in a string is the parity of each character's frequency, each string can be represented as a binary number from 0 to $1023 = 2^{10} - 1$.
- Combining multiple strings can be interpreted as a Bitwise XOR operation between these binary values.

1H. Region Code

- Using two strings with the same binary value always allows you to form a palindrome.
- Other than this case, we only need to consider scenarios where the binary values of the selected strings are all distinct, so let's handle this as an exception.
- The remaining process can be solved using DP.
- We can define $dp[i][j]$ as the minimum sum of lengths when selecting some strings from those with binary values up to i such that their XOR sum is j.
- The size of the DP table is $2^{10} \times 2^{10} = 2^{20}$, and since transitions can be done in $O(1)$, the time complexity is $O(2^{20})$.
- Implement the solution carefully, paying attention to the use of strings with a binary value of 0 and the backtracking process.

11. Treeo

`trees`

Difficulty Intention – **Hard**

- ✓ Statistics:
  - Senior: 16 submissions, 2 correct, 12.500% AC rates
- ✓ First Solve:
  - Senior: **outfinity**[UNIST], 140 min
- ✓ Writer: Leo Jang[`leo020630`]

- First, let's identify all edges that can be removed. This can be achieved through two main approaches.
- 1. Observation + LCA Application
- Let the inverse permutations of A, B, C be P, Q, R.
- For each i $(1 \leq i \leq N)$, any edge belonging to the smallest subtree containing all three vertices P_i, Q_i, R_i cannot be cut. Cutting such an edge would cause i to belong to different subtrees across the three cases.
- Thus, we mark the unusable edges for each i.
- This can be implemented by merging the parent and child vertices of an edge.
- In this process, we need to find the Lowest Common Ancestor (LCA) of the vertices. Since each edge is checked at most once, the time complexity is $O(N \log N)$.

- 2. Direct Set Management
- During a DFS, maintain three sets for A_i, B_i, C_i contained within each subtree.
- Merging these sets can be implemented using the well-known ”Small to Large” technique.
- However, directly comparing sets will result in a Time Limit Exceeded (TLE).
- Instead, manage the XOR sum of each set to check if they match, or track the frequency of each element.
- The time complexity is $O(N \log^2 N)$.

- ✓ There is also a way to avoid direct set management.
- ✓ Assign a random integer within the range $[0, M]$ (where $M \approx 2^{60}$) to each value.
- ✓ Whether two sets match can then be determined by checking if their XOR sums are identical. This technique is known as XOR Hashing or Zobrist Hashing.
- ✓ The time complexity for this approach is $O(N \log N)$.

1I. Treeo

- Now, let's divide the tree into three parts.
- First, if there is one or fewer removable edges, output -1 as it is impossible.
- Otherwise, we can find the answer using binary search.
- Solve the decision problem: "Can we cut the tree into 3 or more subtrees such that every subtree's size is at least X?" This can be solved with Tree DP using a greedy strategy—cut a subtree as soon as its size reaches X or more.
- The maximum X for which the decision problem returns YES is the answer. The time complexity is $O(N \log N)$.
- Alternatively, it can be solved by considering all pairs of removable edges using the Small to Large technique.

11. C) and Query

`segtree`

Difficulty Intention – **Hard**

- ✓ Statistics:
  - Senior: 0 submissions, 0 correct, -% AC rates
- ✓ First Solve:
  - Senior: -
- ✓ Writer: Leo Jang[leo020630]

1I. C) and Query

- Fundamentally, all properties used in the solution of the "C)" problem are applied here.
- Let's define the cost as the number of 'C' characters used as closing parentheses. We will refer to a 'C' used as a closing parenthesis as a **Closing C**.
- **Observation 1.** All 'U' characters appearing after the first Closing C must be Closing Us.
- **Proof:** If an Opening U appears after a Closing C, it is always beneficial to swap their roles.
- Let's define an action called **reduce**. This involves checking if it's possible to change the first Closing C to an opening one and the last Opening U to a closing one, and then performing the change if possible. If the reduce operation succeeds, the cost decreases by 1.

When ’C’ changes to ’U’:

- ✓ 1. If the character was a Closing C:
- ✓ It must remain a Closing U after the change. In this case, the cost decreases by 1.
- ✓ 2. If the character was an Opening C:
- ✓ First, treat it as an Opening U. Then, if there is a Closing U in front of it, swap positions with the first Closing U. Afterward, attempt a **reduce** operation.

When ’U’ changes to ’C’:

- ✓ 1. If the character was an Opening U:
- ✓ It must remain an Opening C after the change.
- ✓ 2. If the character was a Closing U:
- ✓ First, treat it as a Closing C. Then, if there is an Opening C behind it, swap positions with the last Opening C. In this case, the cost increases by 1. Afterward, attempt a **reduce** operation.

- For each query, the cost increases by at most 1. Therefore, the total number of **reduce** operations is $O(N + Q)$.
- All processes can be implemented using data structures like a Lazy Segment Tree to store prefix sums.
- The time complexity is $O((N + Q) \log N)$.
- Alternatively, the problem can be solved by carefully defining the nodes of a Segment Tree.